

Comparação códigos das LLM

Os três códigos implementam análise de tensões em treliças planas pelo método dos nós. Cada versão apresenta filosofia de implementação distinta, com diferentes níveis de robustez e qualidade de código.

1. CÓDIGO GEMINI (codigo.gemini.py) – MELHOR PARA APRENDIZADO PONTOS POSITIVOS: • Conciso e direto: ~100 linhas, fácil de entender • Método de Gauss comentado e referenciado • Visualizações integradas em subplots FRAGILIDADES: • Sistema de reações hardcoded para um caso específico • Não verifica isostasia ($2N = B + \text{reações}$) • Dependência total do NumPy • Sem interatividade VEREDITO: Excelente para demonstração, mas inutilizável para análises gerais.

2. CÓDIGO CLAUDE (codigo.claude.py) – MAIS COMPLETO E PROFISSIONAL PONTOS POSITIVOS: • Arquitetura robusta e modular • Validações abrangentes (comprimentos nulos, sistema singular, isostasia) • Múltiplos modos de entrada (padrão, aleatório, manual) • Implementação sem NumPy (apenas Python nativo) • Gráficos com legendas e salvamento automático • Verificação de equilíbrio (somatório de reações) FRAGILIDADES: • Complexidade elevada (~450 linhas) • Gerador aleatório limitado a grades retangulares VEREDITO: RECOMENDADO para uso real. O mais profissional dos três.

3. CÓDIGO CHATGPT (codigo.chatgpt.py) – FUNCIONAL COM PROBLEMAS DE ESTILO PONTOS POSITIVOS: • Funciona corretamente • Gauss-Jordan com pivotamento completo • Modo interativo FRAGILIDADES GRAVES: • Violações do PEP 8 (estilo de código) • Uso excessivo de `for i in range(len(...))` • Construção ineficiente

de matrizes • Tratamento de reações inconsistente • Sem docstrings ou comentários • Gráficos menos informativos (sem barra de escala)
VEREDITO: Funciona, mas o código é feio e difícil de manter.

RECOMENDAÇÕES PARA VERSÕES FUTURAS 1. Sistema de reações automático (nunca hardcodar) 2. Validação de isostasia antes de resolver 3. Testes automatizados (pytest/unittest) 4. Visualização interativa (matplotlib.widgets) 5. Exportação para CSV/Excel

CONCLUSÃO • Para APRENDER o método dos nós → use o código GEMINI • Para USO PROFISSIONAL/ACADÊMICO → use o código CLAUDE • Para MEIO TERMO → o código CHATGPT resolve, mas o código é frágil Os três códigos demonstram que a mesma funcionalidade pode ser implementada com níveis radicalmente diferentes de qualidade de engenharia de software.

Prompt usado para as LLM

Atue como **Engenheiro de Estruturas** e **Desenvolvedor Python científico**.
Implemente um **programa completo em Python** para **Análise de Trelças Planas Isostáticas**, pronto para rodar no **Google Colab** sem modificações, seguindo rigorosamente os requisitos abaixo:

ENTRADAS

- Número de nós **N** (testar com $N = 4, 8, 16, 32$)
- Coordenadas dos nós: vetores **$x[N]$** e **$y[N]$**
- Número de barras **B**
- Matriz de conectividade **barras[B][2]** (par de nós conectados)
- Vetores de cargas aplicadas **$F_x[N]$** e **$F_y[N]$**
- Vetor de apoios **apoios[N]** com tipos "**livre**", "**movel**" ou "**fixo**"
 - fixo fornece R_x e R_y
 - móvel fornece apenas R_y

PROCESSAMENTO

1. Montar manualmente a **matriz de equilíbrio** (2N equações).
2. Resolver o sistema linear pelo **método de Gauss-Jordan com pivotamento parcial — implementação manual obrigatória** (proibido usar `numpy.linalg.solve`, `scipy` ou qualquer `solve()` pronto).
3. Calcular as **forças internas** em cada barra (tração = positivo, compressão = negativo).
4. Classificar cada barra como **Tração (+)**, **Compressão (-)** ou **Nula**.
5. Calcular estatísticas:
 - barra mais solicitada
 - força média, máxima e mínima
 - somatório das reações
6. Verificar equilíbrio: somatório das reações + cargas externas deve ser zero.

SAÍDAS

1. **Tabela no terminal** com: barra, força (N), classificação, comprimento.
2. **Gráfico 1:** treliça original com nós numerados, barras identificadas e apoios desenhados com símbolos (triângulo fixo, triângulo+base móvel).
3. **Gráfico 2:** mapa de calor — barras coloridas de azul (tração) a vermelho (compressão), espessura proporcional à magnitude; barra de cores com escala em Newtons.
4. **Gráfico 3:** gráfico de barras com a força em cada elemento (azul = tração, vermelho = compressão), valores anotados.
5. **Gráfico 4:** histograma da distribuição das magnitudes das forças, com linha vertical na média.

TRELIÇA-TESTE PADRONIZADA

- **Treliça Warren simétrica:**
 - Nós: (0,0), (1,0), (2,0), (1,1)
 - Barras: 0-1, 1-2, 0-3, 3-2, 1-3
 - Carga: $F_y[1] = -1000$ N (nó central inferior), demais = 0
 - Apoio fixo no nó 0, apoio móvel no nó 2

MENU INICIAL

1. Carregar treliça-teste padronizada
2. Gerar treliça aleatória em grade para N escolhido pelo usuário
3. Entrada manual completa com validação

REQUISITOS DE IMPLEMENTAÇÃO

- Código **modularizado em funções** com comentários explicativos em português.
- Usar **listas/vetores de tamanho variável** definidos pelo usuário.

- Validar todas as entradas (rejeitar negativos, zero onde inválido, texto).
- Instalar dependências automaticamente (`pip install` silencioso).
- Usar backend "`Agg`" do matplotlib para compatibilidade com Colab.
- Salvar cada gráfico como arquivo PNG com nome descritivo e exibir inline com `plt.show()`.
- Mostrar resultados em tabela formatada no terminal.
- Entregar **um único arquivo .py completo**, pronto para rodar no Colab com um único Ctrl+Enter.